

前端开发技术报告

校园二手物品交易平台——转转

答辩人：李硕

角色：前端开发工程师

技术栈：Vue3 + TypeScript + Vite + Element Plus + Pinia +
Axios + ECharts

软件工程实训 · 课程设计答辩

华中农业大学 · 2026 年 7 月

目录

1 个人角色与工作概述	2
2 技术栈说明	2
3 首页开发	2
3.1 Hero 横幅	2
3.2 公告栏滚动轮播	3
3.3 商品分类导航	3
3.4 商品网格	3
4 商品模块	3
4.1 商品列表页	3
4.2 商品详情页	4
4.3 商品发布页	4
5 购物车模块	4
6 订单列表页	5
7 管理后台	5
7.1 AdminLayout 布局	5
7.2 AdminUsers 用户管理页	5
7.3 AdminProducts 商品审核页	6
7.4 AdminOrders 订单管理页	6
7.5 AdminAnnouncements 公告管理页	6
8 ECharts 数据可视化	6
8.1 分类占比饼图	6
8.2 每日趋势折线图	6
9 种子数据与图片方案	7
9.1 155 件商品数据编写	7
9.2 自定义产品图片生成方案	7
10 代码贡献统计	7
11 核心技术亮点	7

12 商品列表筛选系统设计	8
13 管理后台系统架构	9
14 ECharts 图表实现详解	9
14.1 饼图实现	9
14.2 折线图实现	10
14.3 内存泄漏防护	10
15 购物车状态管理	10
16 商品发布页的图片上传	10
17 种子数据与图片方案的迭代过程	11
18 响应式设计实现	11
19 项目中的技术决策	12
20 遇到的问题与解决方案	12

个人角色与工作概述

在本次软件工程实训项目中，我担任前端开发工程师，主要负责以下模块：

- **首页开发**：Hero横幅、实时统计数据、公告栏滚动轮播、商品分类导航、商品网格展示、时间格式化
- **商品模块**：商品列表页（多条件筛选/分页/排序）、商品详情页（图片画廊/成色标签/收藏）、商品发布页（多图上传/级联分类选择/表单校验）
- **购物车模块**：购物车列表、全选/取消、数量调节、结算栏
- **订单列表页**：状态 Tab 切换、买家卖家视角切换、订单卡片展示
- **管理后台**：5个管理页面（用户管理、商品审核、订单管理、数据统计、公告管理）+ AdminLayout 布局
- **数据可视化**：ECharts饼图（分类占比）+ 折线图（每日趋势）
- **种子数据与图片**：155件商品的原始数据编写、自定义产品图片生成方案

技术栈说明

前端基于 Vue 3 Composition API + TypeScript 架构，使用 Vite 5 构建，Element Plus 作为 UI 组件库，Pinia 状态管理，Axios 封装 HTTP 请求。管理后台统计图表使用 ECharts 5 实现数据可视化。

Vue 3 的 Composition API 通过 `setup()` 语法糖将响应式状态、计算属性和方法集中管理，在开发复杂的商品筛选、购物车状态管理等功能时，代码组织更加清晰。TypeScript 的类型系统在前后端接口对接中起到了关键作用，API 返回数据的结构通过类型定义形成明确的契约。

首页开发

3.1 Hero 横幅

首页顶部实现了全宽渐变绿色背景的 Hero 横幅，包含以下元素：

渐变背景：使用 CSS `linear-gradient(135deg, #1B4332, #2D6A4F, #40916C)` 实现三色渐变。配合三个浮动圆形装饰元素（使用 CSS Animation `@keyframes float` 实现上下浮动动画），营造动态视觉氛围。

品牌标语：使用 ZCOOL QingKe HuangYou 字体（Google Fonts）渲染“淘你所爱转你所闲”的品牌标语，配合 CSS `letter-spacing` 实现字间距效果。

实时统计数据: 从后端 `/index/statistics` 接口获取注册用户数、在售商品数、成交订单数三个关键指标, 使用大号数字加单位标签展示, 数值使用 `toLocaleString()` 添加千分位分隔符。

3.2 公告栏滚动轮播

使用纯 CSS Animation 实现了公告栏的水平滚动轮播效果。公告标题在一行内循环滚动, 鼠标悬停时暂停动画 (`animation-play-state: paused`)。

核心原理: `@keyframes noticeScroll { 0% { transform: translateX(0) } 100% { transform: translateX(-50%) } }`。通过将公告内容复制一份并拼接在末尾, 实现无缝循环。外层容器使用 `mask-image: linear-gradient(to right, transparent 0%, black 4%, black 96%, transparent 100%)` 实现边缘渐隐效果。

3.3 商品分类导航

使用 Element Plus 的 `el-tabs` 组件风格, 实现了 8 个分类标签的导航栏。每个标签包含 Emoji 图标和分类名称, 点击切换加载对应分类商品。选中态使用主色高亮 (变为主色背景 + 白色文字), 配合 `box-shadow` 实现发光效果。

3.4 商品网格

使用 CSS Grid 布局 `grid-template-columns: repeat(auto-fill, minmax(240px, 1fr))` 实现响应式商品网格。每张商品卡片包含封面图、分类标签、标题、价格、浏览量和时间信息。

卡片交互动效: `hover` 时卡片上浮 4px + 增加阴影 + 封面图放大 1.08 倍。使用 `transition: all 0.3s ease` 实现平滑过渡。

相对时间格式化: 实现了“刚刚 / X 分钟前 / X 小时前 / X 天前 / 日期”的时间显示逻辑。核心判断: 60 秒内 → 刚刚, 1 小时内 → X 分钟前, 24 小时内 → X 小时前, 超过 24 小时 → 显示日期。

商品模块

4.1 商品列表页

商品列表页是多条件筛选的综合交互页面, 实现了以下筛选功能:

关键词搜索: 输入框传入 `keyword` 参数, 后端使用 MySQL FULLTEXT 全文索引进行标题和描述搜索。

分类筛选: 使用 Element Plus Cascader 级联选择器, 支持一级分类 (电子产品/书籍教材等 6 个) + 二级分类 (手机/电脑/平板等 20 个)。

价格区间：minPrice 和 maxPrice 两个参数过滤。

商品成色：下拉选择：全新/几乎全新/轻微使用痕迹/明显使用痕迹。

排序方式：价格（升序/降序）、发布时间、浏览量。排序参数通过后端 MyBatis XML 中白名单校验后使用 `${sort}` 动态 SQL 拼接。

URL 参数同步：所有筛选参数同步到 URL query string，用户刷新页面后筛选条件保持不变。通过 `route.query` 读取初始参数，`router.push` 更新 URL。

分页组件：使用 Element Plus Pagination，每页 20 条，支持页码和每页条数切换。

4.2 商品详情页

图片画廊：主图大图展示 + 底部缩略图列表。点击缩略图通过 `currentImage.value = img.url` 切换主图。图片加载失败时使用 CSS 占位符。

成色标签：根据商品成色（全新/几乎全新/轻微使用痕迹/明显使用痕迹/老旧）显示不同颜色的标签——绿色（全新）、蓝色（几乎全新）、橙色（轻微痕迹）、紫色（明显痕迹）。

HTML 安全过滤：商品描述可能包含富文本 HTML。我实现了 `sanitizedDescription` 计算属性，使用三重防线过滤：正则移除 `<script>` 和 `<style>` 标签 剥离所有 HTML 标签 使用 DOM textarea 解码 HTML 实体。这有效防止了 XSS 攻击。

封面图回退机制：当商品封面图加载失败时，使用 `getCategoryCover` 工具函数根据分类名称返回对应的 Emoji 占位符号（如手机 → 、书籍 → ），比纯灰色占位图更友好。

4.3 商品发布页

分类级联选择器：使用 Element Plus Cascader 实现两级分类选择，父级可选电子产品/书籍教材等 6 个大类，展开后显示对应的子分类。

多图上传：使用 El-Upload 的 picture-card 模式，支持 JPG/PNG/GIF/WebP 格式，单文件 5MB 限制。图片上传后即刻预览，支持删除和排序。

表单校验：使用 Element Plus Form 的 `rules` 实现前端校验——标题必填且最长 50 字符、分类必选、价格必填且为正数。

购物车模块

购物车实现了完整的选择-修改-删除-结算功能：

复选框选择：每个购物车项有独立复选框，支持全选/取消全选。使用 `computed` 属性动态计算全选状态。

数量调节：使用 El-Input-Number 组件，范围 1-99。数量变化时使用防抖策略（300ms）向后端同步，避免用户快速连续点击导致的频繁请求。

底部结算栏:显示”已选 N 件”和合计金额,合计金额使用 `filter(i=>i.selected).reduce((s,` 实时计算。

空状态引导:购物车为空时展示 El-Empty 组件,引导用户去商品列表浏览。

订单列表页

状态 Tab 切换:6 个 Tab (全部/待付款/待发货/待收货/已完成/已取消),点击 Tab 传入对应的 `status` 参数重新查询。

买家卖家视角切换:顶部增加 El-Radio-Group 切换按钮——”我买的”/”我卖的”。这是订单列表页最重要的改进之一。原始代码写死了 `role:'buyer'`,卖家永远看不到自己的待发货订单。我修复为使用 `ref('buyer')` 绑定到 Radio Group,切换时自动重新加载数据。

卖家待发货提示:当切换到卖家视角时,显示绿色提示条引导卖家查看待发货订单;在待发货 Tab 下提示”这些是买家已付款等待你发货的订单,点击进入订单详情即可录入物流”。

”去发货”按钮:卖家待发货订单上显示一个突出的”去发货”按钮,一键跳转到订单详情页。

状态映射函数:封装 `statusType()` 函数,将订单状态映射为 Element Plus Tag 类型 (`warning/primary/success/info/danger`),在 AdminProducts、AdminOrders、OrderList、OrderDetail 四个页面中复用。

管理后台

7.1 AdminLayout 布局

使用 Element Plus 的 `el-menu` 组件实现侧边栏导航。包含 5 个菜单项:用户管理、商品审核、订单管理、数据统计、公告管理。使用 `router` 属性实现菜单项与路由的自动关联。

路由鉴权在 `router.beforeEach` 中实现:检查 `meta.requiresAdmin` 标志,读取 `localStorage` 中的用户信息,非管理员自动重定向到首页。

侧边栏使用 CSS `@media` 查询实现响应式折叠,小屏下自动隐藏或收缩。

7.2 AdminUsers 用户管理页

表格展示所有用户的 ID、用户名、邮箱、手机号、角色、信誉分、状态、注册时间。操作列包含”启用/禁用”和”切换角色”两个按钮。使用确认对话框防止误操作。分页功能通过后端参数实现。

7.3 AdminProducts 商品审核页

状态筛选栏支持待审核/在售/已下架/审核驳回四种状态。操作按钮包括”通过”（状态改为在售）、”驳回”（状态改为审核驳回）、”下架”（强制改为已下架）、”编辑”（弹出编辑对话框，可修改标题/价格/成色/状态/描述）。

分页组件在筛选状态变化时自动重置到第一页（`watch` 监听筛选参数变化）。

7.4 AdminOrders 订单管理页

表格展示订单号、商品名、买家、卖家、金额、状态、创建时间。支持状态筛选和订单号搜索。操作列根据当前状态显示对应按钮：待付款 → 标记已付款、待发货 → 标记已发货、待收货 → 标记已完成。

订单详情弹窗使用 `El-Dialog` 展示完整订单信息，包括收件人、地址、物流信息等。

7.5 AdminAnnouncements 公告管理页

公告列表表格展示标题、状态（已发布/草稿）、发布时间。新增/编辑弹窗包含标题输入和内容文本区。删除操作带确认对话框。

ECharts 数据可视化

8.1 分类占比饼图

使用 ECharts 5 的 Pie Chart 实现环形图，展示 20 个商品分类的商品数量分布。数据从后端 `/admin/statistics/detail` 接口获取。

核心技术点：

- 图表初始化使用 `echarts.init()` 绑定到 `ref` 引用的 DOM 元素
- 通过 `window.addEventListener('resize', ...)` 监听窗口变化,调用 `chart.resize()` 实现自适应
- 在 `onUnmounted` 生命周期中调用 `chart.dispose()` 释放资源，防止内存泄漏

8.2 每日趋势折线图

使用 ECharts Line Chart 展示用户注册量和订单成交量的每日趋势。双 Y 轴设计，分别展示用户数和订单数。X 轴为日期，使用 `category` 类型。Tooltip 在鼠标悬停时显示具体数值。

种子数据与图片方案

9.1 155 件商品数据编写

我编写了覆盖全部 20 个商品分类的 155 件商品种子数据。每件商品包含：标题（品牌 + 型号 + 规格）、描述（成色 + 配置 + 特点）、合理价格和原价、正确成色（5 个枚举值）、随机浏览量和收藏数。

商品覆盖手机（15 件）、电脑（15 件）、平板（10 件）、耳机（12 件）、相机（8 件）、配件（10 件）、教材教辅（7 件）、文学小说（5 件）、科技科普（4 件）、考试资料（4 件）、宿舍用品（8 件）、厨房电器（6 件）、美妆个护（6 件）、上衣（7 件）、裤装（4 件）、鞋类（6 件）、箱包（3 件）、运动器材（9 件）、户外装备（7 件）、其他（9 件）。

9.2 自定义产品图片生成方案

经过多轮迭代（LoremFlickr → Picsum.photos → Unsplash → Pexels），最终采用 Python + Pillow 自定义生成图片的方案。每张商品图片显示该商品的分类标签和商品名称，使用分类对应的品牌色作为背景，确保图文 100% 匹配。

核心原理：Python 脚本从数据库导出商品列表（ID + 标题 + 分类），使用 Pillow 库为每个商品生成 640x480 的 JPG 图片。图片包含分类标签（大写英文）、商品标题（自动换行）、品牌标识文字。图片存储到本地 uploads/products/ 目录，通过 Nginx 代理由 Docker 容器中的后端服务提供访问。

```
for pid, title, cat_id in products:
    color, label = cat_colors.get(cat_id)
    img = Image.new('RGB', (640, 480), rgb)
    draw = ImageDraw.Draw(img)
    draw.text((320, 240), title, fill="white", anchor="mm")
    img.save(f"{out}/{pid}.jpg", quality=90)
```

这确保了每件商品的图片与商品完全对应，不存在任何图文不符的问题。

代码贡献统计

核心技术亮点

1. **级联分类选择器**：使用 Element Plus Cascader 实现二级分类选择，父分类可展开查看子分类，选择体验流畅。支持动态加载和默认值回填。
2. **ECharts 可视化**：饼图展示分类占比、折线图展示时间趋势，图表自适应窗口大小变化。在组件卸载时调用 `dispose()` 释放图表实例，防止内存泄漏。

模块	代码行数 (估计)
首页 (HomePage)	约 730 行
商品模块 (ProductList/Detail/Publish/MyProducts)	约 1,100 行
购物车与订单列表 (CartPage/OrderList)	约 500 行
管理后台 (AdminLayout+5 管理页)	约 1,300 行
工具与配置 (productImage/router/types)	约 200 行
SQL 种子数据	约 300 行
自定义图片生成脚本	约 50 行
合计	约 4,180 行

- 分类占位图回退机制**: 商品封面图加载失败时根据分类名称返回对应 Emoji 占位封面, 比纯灰色占位图更友好。
- HTML 内容安全净化**: 商品描述使用正则移除 script/style 标签后剥离所有 HTML 标签, 再通过 DOM textarea 解码 HTML 实体, 三重防线防止 XSS 攻击。
- 全功能管理后台**: 涵盖用户管理、商品审核、订单管理、数据统计、公告管理五大模块, 支持搜索、筛选、分页、编辑、状态流转全流程操作。
- 响应式设计**: 所有页面适配移动端 (768px), 商品网格从多列自适应为双列, 导航栏和侧边栏在小屏下自动隐藏或折叠。
- 订单列表双视角切换**: 修复了原始代码中写死 buyer 角色导致卖家无法看到订单的问题, 通过 Radio Group 实现买卖双方视角的自由切换。
- 自定义图片方案**: 经过 6 轮迭代, 最终采用 Python+Pillow 自定义生成图片的方案, 实现了图文 100% 匹配, 所有图片存储于本地, 无需依赖任何外部服务。

商品列表筛选系统设计

商品列表页的筛选系统涉及前后端协同的复杂参数传递。核心设计原则是: 所有筛选状态通过 URL query parameters 持久化, 确保页面刷新后筛选条件不丢失。

参数传递流程:

- 用户操作筛选控件 (输入关键词、选择分类、设置价格区间、选择成色、切换排序) → 更新组件响应式状态
- 调用 `router.push({query: params})` 更新 URL (不刷新页面)
- `watch` 监听 `route.query` 变化, 触发 `loadProducts()` 重新拉取数据

4. 页面初次加载时从 `route.query` 读取初始参数值

分类级联选择器的技术细节：Element Plus 的 `el-cascader` 组件接受一个树形结构的 `options` 数组。后端 `/category/list` 接口返回的是嵌套 JSON 结构（一级分类包含 `children` 数组），前端直接传入 Cascader 即可使用。选中值为数组 `[parentId, childId]`，提交时取最后一个元素作为 `categoryId`。

排序白名单：后端 MyBatis XML 中使用 `${sort}` 动态 SQL 拼接 `ORDER BY` 子句，为防止 SQL 注入，前后端都实现了白名单校验——只允许 `price`、`created_at`、`view_count`、`favorite_count` 四个字段和 `asc`、`desc` 两个方向。

管理后台系统架构

管理后台的 5 个页面共享一套设计模式：表格 + 筛选栏 + 操作按钮 + 分页 + 对话框。我抽象了这一模式，使得每个管理页面的开发只需关注数据和操作逻辑的差异。

AdminLayout 的布局结构：使用 Element Plus 的 `el-container` + `el-aside`（侧边栏）+ `el-main`（内容区）。侧边栏使用 `el-menu` 组件的 `router` 模式，菜单项的 `index` 属性直接对应路由路径，点击自动导航。

分页组件的通用处理：每个管理页面都维护 `page` 和 `size` 两个响应式变量。筛选条件变化时自动将 `page` 重置为 1。后端返回的数据包含 `total` 字段，前端据此计算总页数。

乐观更新 vs 确认后更新：对于状态切换操作（启用/禁用、通过/驳回），采用确认对话框（`El-MessageBox.confirm`）确保用户明确意图后再执行操作，避免误操作。

ECharts 图表实现详解

14.1 饼图实现

分类占比饼图使用 ECharts 的 `Pie` 类型。关键配置：

- `series.type = 'pie'`：指定图表类型为饼图
- `series.radius = ['40%', '70%']`：设置为环形图（内径 40%，外径 70%）
- `series.label`：显示分类名称和百分比
- `series.emphasis`：hover 时扇区放大和阴影效果
- `tooltip`：鼠标悬停时显示分类名称、商品数量、占比

数据转换：后端返回的数据格式为 `{name: string, value: number}[]`，前端直接传入 ECharts 的 `series.data`。

14.2 折线图实现

每日趋势折线图使用双 Y 轴设计。左 Y 轴表示用户注册量，右 Y 轴表示订单成交量。X 轴为日期。两条折线使用不同颜色区分。

自适应实现：在 `onMounted` 中通过 `window.addEventListener('resize', handleResize)` 监听窗口尺寸变化，回调函数中调用 `chart.resize()` 重绘图表。在 `onUnmounted` 中移除事件监听器并调用 `chart.dispose()` 释放图表资源。

14.3 内存泄漏防护

ECharts 实例持有对 DOM 元素的引用。如果不在组件销毁时手动释放，即使 DOM 已被 Vue 移除，ECharts 实例仍然存在并占用内存。在单页应用（SPA）中，用户频繁切换页面会导致内存持续增长。本项目中每个使用 ECharts 的组件都在 `onUnmounted` 中调用 `chart.dispose()`，这是 SPA 中使用图表库的标准实践。

购物车状态管理

购物车的数据流设计：购物车列表从后端 `/cart/list` 接口获取，每个购物车项包含商品信息、数量、是否选中三个核心字段。

选中状态管理：每个购物车项有独立的 `selected` 字段。全选/取消全选通过遍历列表设置每个项的 `selected` 实现。全选状态通过 `computed` 属性动态计算——当所有项的 `selected` 都为 `true` 时，全选复选框显示为选中状态。

数量修改防抖：使用 JavaScript 的防抖模式避免用户快速连续点击数量调节按钮导致的频繁 API 调用。核心原理：每次数量变化时清除之前的定时器，设置新的 300ms 定时器，只有最后一次操作在 300ms 内没有后续操作时才会真正发送 API 请求。这是一个经典的性能优化模式。

结算逻辑：合计金额 = 所有选中项的 `price * quantity` 之和。使用 `Array.reduce` 实现。点击结算前检查是否有选中项，如果未选择任何商品则提示用户。

商品发布页的图片上传

商品发布页的多图上传功能是用用户体验的关键环节。使用 Element Plus 的 `el-upload` 组件配合 `picture-card` 模式。

上传流程：用户选择文件 → 前端校验文件类型（JPG/PNG/GIF/WebP）和大小（5MB）→ 调用 `/upload/image` 接口上传 → 后端存储文件并返回图片 URL 和 ID → 前端显示预览缩略图 → 发布商品时将图片 ID 列表一并提交。

预览与删除：`el-upload` 的 `picture-card` 模式在文件选择后自动显示缩略图预览。上传成功后调用 `handleSuccess` 回调更新图片列表。删除操作通过 `handleRemove`

回调从图片列表中移除对应项。

种子数据与图片方案的迭代过程

商品种子数据和图片方案经历了多次迭代，每次迭代都基于上一版本的问题进行改进：

第一版——LoremFlickr：使用关键词匹配的 Flickr 图片。问题：当某个关键词没有匹配结果时，返回默认的“熊”占位图，严重影响体验。且同一关键词每次请求返回不同图片，商品图片不稳定。

第二版——Picsum.photos：使用 Lorem Picsum 的随机图片。问题：图片是随机的自然风光、建筑等非产品图片，与商品完全无关。

第三版——Unsplash 验证图片：手动筛选并验证 18 个可用的 Unsplash 产品类图片。问题：图片与实际商品不完全匹配（如 iPhone 显示的是某人的手机照片，而非 iPhone 产品图）。且部分 Unsplash URL 后续失效（返回 404）。

第四版——Apple/Xiaomi CDN：直接使用苹果和小米官方 CDN 的产品图片。问题：只有部分品牌提供，无法覆盖全部 20 个分类。且 CDN URL 不保证永久有效（如 iPad 相关 URL 在后续测试中返回 404）。

第五版——Pexels：使用 Pexels 免费图库下载图片。问题：下载的图片可能包含人物或不相关场景，与商品标题不符。

第六版（最终版）——Python + Pillow 自定义生成：使用 Python 脚本从数据库读取商品信息，为每个商品生成 640×480 像素的专属图片。图片显示商品分类标签和商品名称，使用分类对应的品牌色作为背景。这一方案实现了图文 100% 匹配，所有图片存储于本地文件系统，通过 Docker 数据卷持久化，完全不依赖任何外部服务。

每一版迭代都解决了前一版的痛点，最终版方案在生产环境中表现稳定，155 件商品图片全部本地存储，访问延迟接近于零。

响应式设计实现

本项目面向 PC 端用户，但也考虑了移动端的适配。核心响应式策略：

商品网格：使用 CSS Grid 的 `repeat(auto-fill, minmax(240px, 1fr))` 实现列数自动适配。在大屏上可以显示 5-6 列，中屏 3-4 列，小屏（768px）2 列。

导航栏：在大屏上显示完整的水平导航栏；在小屏上可以折叠为汉堡菜单（使用 CSS `@media` 查询和 `display:none/block` 控制）。

管理后台侧边栏：在大屏上默认展开；在小屏上默认折叠，点击按钮展开。

字体大小：使用相对单位（rem、百分比）而非固定像素值，确保在不同分辨率下文字大小适中。

项目中的技术决策

为什么首页使用 CSS Animation 而非 JavaScript 动画：CSS Animation 由浏览器的合成器线程处理，不占用主线程资源。在滚动、动画等场景中性能优于 JavaScript 动画。对于公告栏滚动这种简单重复动画，CSS Animation 是最佳选择。

为什么管理后台使用表格而非卡片布局：管理后台需要展示大量结构化数据（用户列表、订单列表等），表格是最高效的展示方式。Element Plus 的 `el-table` 组件提供了排序、筛选、固定列等高级功能。

为什么使用 El-Dialog 而非路由跳转：管理后台的编辑操作使用对话框(El-Dialog)，而非跳转到独立的编辑页面。对话框的优势在于保持用户在当前页面的上下文，编辑完成后立即刷新表格数据，无需页面导航。

遇到的问题与解决方案

问题一：ECharts 图表不随窗口 resize 自适应。ECharts 默认不会监听窗口尺寸变化。解决：手动添加 `window.addEventListener('resize', handler)` 并在 handler 中调用 `chart.resize()`。

问题二：管理后台分页在筛选条件变化时不重置。用户在第一页设置筛选条件后，如果当前页码大于筛选后的总页数，会显示空页。解决：使用 `watch` 监听筛选参数变化，在回调中设置 `page = 1`。

问题三：商品图片加载失败导致页面布局混乱。如果图片 URL 失效，`<img>` 标签显示为空白区域，破坏了卡片布局。解决：实现 `@error` 事件处理器，加载失败时替换为 Emoji 占位图。

问题四：购物车结算后创建多个独立订单。原始代码中购物车结算时对每个选中项分别调用 `orderApi.create`，这些调用是并行的，可能导致部分成功部分失败。虽然当前版本的模拟支付不需要事务性保证，但在真实场景中应该使用后端批量创建订单的接口。这个问题已在代码中标注为待优化项。

问题五：首页骨架屏与实际内容高度不一致。骨架屏的灰色占位卡片与实际商品卡片高度不一致时，数据加载完成后的替换会产生视觉跳动。解决：骨架屏卡片使用与实际卡片相同的布局结构（图片区域 + 文字区域），确保高度一致。

问题六：分类数据中的 Emoji 在不同操作系统显示不一致。Windows、macOS、iOS 的 Emoji 渲染效果不同。解决：使用 SVG 图标或 Font Awesome 等跨平台一致的图标方案替代 Emoji。这一改进已纳入后续优化计划。